

Erin Roper

Hw 1

3)

~~def connect~~

(network(connectors(connection))
nodes map {str, int} computers

```
for connection in connectors
    for comp in connection
        if comp in computers
            computers[comp] += 1
```

else

if computers[comp] == 1

str trusted = connectors[0][0] ← and value doesn't matter

for [key, value] in computers

if value > computers[trusted]

trusted = key

new_connectors = connectors

for connection in new_connectors

if trusted in connection

time cost: $2n + 2n + n + T(n \frac{n-1}{2})$
if not connection: remove(connection)

if not new_connectors: empty

return check_connection(new_connectors).append(trusted)

yes, else return [trusted]

time cost: $2n + 2n + n + T(n \frac{n-1}{2})$

$2n + 2n + n + n^2$ (approx.)

$O(n^2)$

yes, O(n^2)

No, does not always work

$(A, B) (A, C) (A, D) (B, E) (C, F) (D, G)$

this case pulls A, then G, F, E
it's 3, 1, 1, 1

Smallest B really B, C, D
it's 2, 2, 2

most connections is most trusted,
but second most connections could overlap
with most connections, ideal order is

$A, B=C=D, E=F=G$